5章
サービスとAPI管理

ほとんどの組織は、データメッシュを望んでいるのではありません。**データと統合のメッシュ**、つまり分析面と運用面の両方を全体的に管理するという統一されたアプローチを求めているのです。なぜでしょうか。2つのプレーンを整合させることで得られるメリットが大きいからです。運用データアーキテクチャは分析データアーキテクチャと対照的です。なぜなら、運用プレーンはコマンドを処理し、予測と小さなデータセットのリアルタイム処理が必要です。これに対し、分析プレーンはデータの読み出しに重点を置き、大きなデータセットを使用し、それほど時間的な制約がないような複雑なデータ分析が必要だからです。しかし、（データのように）APIをどのようにプロダクトとして扱うべきか、アプリケーションとドメインの境界をどのように設定すべきかといったドメインモデルには、多くの重なりがあります。ビジネスケイパビリティは、運用や分析の視点でアーキテクチャを見ても同じものです。これらの機能を提供するアプリケーションの背後には、それらを管理し、稼働させるチームがいます。チームが開発に使用する言語も同じです。アプリケーションの設計に影響を与える一意のコンテキストは、データプロダクトやAPIの設計に影響を与えるコンテキストと一致します。さらに、API管理において、アプリケーションの疎結合化や自律的でアジャイルなチームへのシフトは、データメッシュアーキテクチャへのシフトと似ています。したがって、データ管理とAPI管理に同様のベストプラクティスを適用することは理にかなっています。

この章ではAPI管理、サービス指向アーキテクチャ（SOA）、そのほか多くのリアルタイム統合とデータパターンを取り上げます。すべてのデータ専門家がAPI管理に精通しているわけではありません。このため、最初の節では、SOAとAPIが何なのかについて理解してもらいたいと思います。データ管理の分野においてAPI管理から学べることはたくさんあります。次に、過去10年間のベストプラクティスと、複雑な統合レイヤーを構築する際に生じるスケーラビリティの問題について見ていきます。最後に、ドメイン駆動設計とデータプロダクトの管理を関連づけ、その観点からAPIをどのように管理すべきかを検討し

ます。この章を読み進めば、最新のサービス指向アーキテクチャをしっかりと理解できます。このアーキテクチャを活用すれば、分散型ソフトウェアシステムやリアルタイムソフトウェアシステムのスケーラビリティが実現できます。また、このアーキテクチャが大局的に見て、4章で説明したデータプロダクトアーキテクチャとどのように関連しているかについても学べます。

API管理について説明する前に、データ管理の本の中でAPIとサービス管理の章があることを不思議に思われているのではないでしょうか。まず、APIは正確に最新のデータにアクセスするために不可欠です。APIはデータプロダクトとは異なります。データプロダクトはデータのコピーであり、その性格上非同期です。APIが必要不可欠であるのは、機械学習サービスなどのリアルタイム分析サービスへのインタフェースとなるからです。また、APIは1章で取り上げた運用システムと分析システムの統合にも使われます。最後に、APIと統合パターンは、（例えば、データカタログの統合やデータ契約情報の送信など）複雑なデータ管理エコシステムを設計する際に重要な役割を果たします。

5.1　API管理の紹介

APIを深く掘り下げる前に、まずAPIとSOAの基本、そして統合に関する課題について説明しましょう。

API（**application programming interface**）は、アプリケーションや、ソフトウェアコンポーネント、サービスが直接通信できるようにするためのものです。お互いのデータにアクセスしたり、リアルタイムに処理を開始したりしなければならない、運用システムや、トランザクションシステム、分析システムは、APIパターンの好例です。また、クラウド企業や、外部企業、ソーシャルネットワークとのやり取りでもAPIパターンが使われます。さらに、APIは、企業がデジタルチャネルで豊かなユーザーエクスペリエンスを提供したい場合にも、重要な役割を果たします。例えば、旅行代理店では、そのWebサイトにアクセスしたユーザーに対して、航空会社のフライトやスケジュールをリアルタイムで確認できるようにしたいと考えるでしょう。

API通信の大部分は同期型の**リクエストレスポンスパターン**を使用しています。つまり、（サービスの）リクエスト側が、応答側システムにリクエストを送信し、応答側システムがリクエストを受信して処理を行い、レスポンスとしてメッセージを返します。リクエスト側はこの応答を待ってから処理を続けます。一般的に、交換されるデータ量は少なく、低遅延のユースケースで必要と

なるリアルタイム通信を実現可能にします。別のパターンとして、非同期データ通信があります。非同期データ通信では、処理を継続するのに、即時の応答を必要としません。送信されるデータは非同期で、また、時間をおいて配信されます。APIを非同期で使用し、イベント駆動型処理と組み合わせることもできます。例えば、APIに対して、HTTPを使って送信されたメッセージをメッセージキューに格納しておき、後で処理してもかまいません。このようなモデルは、**出版・購読型（publish and subscribe**、pub/sub）モデルと呼ばれています。

この章では、同期型のリクエスト/レスポンス通信に的を絞って説明します。6章では、イベント駆動型処理、キューイング、非同期通信、そしてこれらのパターンがSOAやAPIとどのように組み合わされるのかについて説明します。

API管理はサービス指向アーキテクチャと密接に関係しています。SOAはソフトウェア開発とアプリケーションの接続方法に関する最新のアプローチです。この考え方は10年以上前に生まれ、今日でも広く使われています。「SOAは時代遅れだ」と主張するアーキテクトやエンジニアもいますが、マイクロサービス、機械学習とリアルタイム意思決定、ビジネスプロセス管理、クラウドAPI、オープンAPI（https://oreil.ly/QEqiD）、SaaSなどのトレンドにより、実際にはこれまで以上に活用されています。

5.2　サービス指向アーキテクチャとは

SOAについて調べると、さまざまな意見を見かけます。実際、著者であり開発者でもある Martin Fowler氏は、SOA（https://oreil.ly/cAN9H）は人によって意味が異なるため、説明することは不可能だと考えています。

筆者にとってのSOAとは、（Webベースの）サービスを介したアプリケーション間の通信を実現するデザインパターンです。これは**ビジネス機能**を**サービス**や**API**の形式で公開することです。SOAは、アプリケーションの複雑さを抽象化して隠すためにも使われます。複雑さを見せるかわりに提供されるものはビジネス機能です。

The Open Groupは2009年、後に**SOAソースブック**（https://oreil.ly/VdrB4）となるホワイトペーパーにおいて、**サービス指向アーキテクチャ**という言葉を生み出しました。The Open GroupはSOAを「サービス指向をサポートするアーキテクチャスタイル」と定義しています。サービス指向とは、サービスとサービスベースの開発、およびサービスの成果物から考える方法です。

SOAでは、アプリケーションの通信は、SOAP（https://oreil.ly/QbkfU）やJSON（https://oreil.ly/k4LHu）などの標準プロトコルや、REST（次のコラムで紹介）などの一般的なソフトウェアアーキテクチャスタイルを介して行われます。SOAでは、リアルタイムの同期通信（返事が届いてから処理を継続する）と、非同期通信（返事が届くのを待たない）があります。

SOAは、また、アプリケーションとドメインの間の境界を明確にして、分離することでもあります。SOAは、ほかのアプリケーションを変更することなく、アプリケーションを個別に変更できるようにします。このため、SOAは、アプリケーションの開発を高速化し、複雑なアプリケーションランドスケープを保守するために利用できます。

リソースと操作

REST（Representational State Transfer）は、一連の制約と抽象化の原則を定義したアーキテクチャスタイルです[†1]。Webサービスを作成するときに利用します。RESTにおいて重要な概念は、**リソース**（https://oreil.ly/lk2oo）です。リソースとは、それ自体がモノとして抽象化・参照される重要なものを意味します。一意に識別できるオブジェクトや情報源は、リソースになり得ます。例えば、顧客や、契約、アカウント、注文、製品などです。RESTful APIでは、これらのリソースと対話するためのHTTPプロトコルが設定されており、データベースのCRUD（作成《create》、読み出し《read》、更新《update》、削除《delete》）操作を真似ています。

POST
新しいリソースを作成するための**create**（作成）メソッド。
GET
1つまたはすべてのリソースを取得するための**read**（読み出し）メソッド。
PUT
既存のリソースを置き換えるための**update**（更新）メソッド。リソース内の特定のフィールドのみを更新する場合は、**PATCH**を使用するのが一般的です。
DELETE
特定のリソースの削除を行う**delete**（削除）メソッド。

RESTがAPIのためのアーキテクチャスタイルであるのに対し、CRUDはデータ操作のための対話スタイルです。

[†1] Roy Fielding氏は、2000年の博士論文“Architectural Styles and Design of Network Based Software Architectures（ネットワークベースのソフトウェアアーキテクチャのアーキテクチャスタイルと設計）”（https://oreil.ly/J7B1r）でRESTを定義しました。

アーキテクトやエンジニアの中には、SOAはビジネス機能やプロセスを提供するものであり、データにはあまり関係がないと主張する人もいます。これはある意味では正しいのですが、コアとなる要素は常にデータです。ネットワークが情報を運ばなければ、通信は成り立ちません。SOAも機能しません。SOAは、このような理由から、データと非常に関係のあるものなのです。

また、SOAは運用システムやトランザクションシステムの領域でのみ使われると主張するアーキテクトやエンジニアもいるでしょう。筆者はこの表現は正確だとは思いません。なぜなら、SOAは運用と分析の世界のアプリケーションをリアルタイムで接続するアーキテクチャだからです。例えば、機械学習モデルを訓練し、リアルタイムのエンドポイントとしてコンシューマに提供するとします。そのためには、サービス間接続、つまりAPI接続が必要です。

SOAの革新的な点としては、抽象化も挙げられます。機能とデータはサービスによってカプセル化されます。アプリケーションがお互いを直接起動したり呼び出したりするのではなく、図5-1に示されているように、標準的な相互運用

パターンを使用して、明確に定義されたサービスインタフェースを介してデータを交換したり、機能を提供したりします。サービスプロバイダーのアプリケーション内部の複雑さとデータは隠蔽されたままであり、サービスコンシューマは気にする必要がありません[†2]。

[†2] このトピックについては、Pat Helland氏の論文“Data on the Outside Versus Data on the Inside（外側のデータと内側のデータ）”（https://oreil.ly/oOH6r）を参照してください。

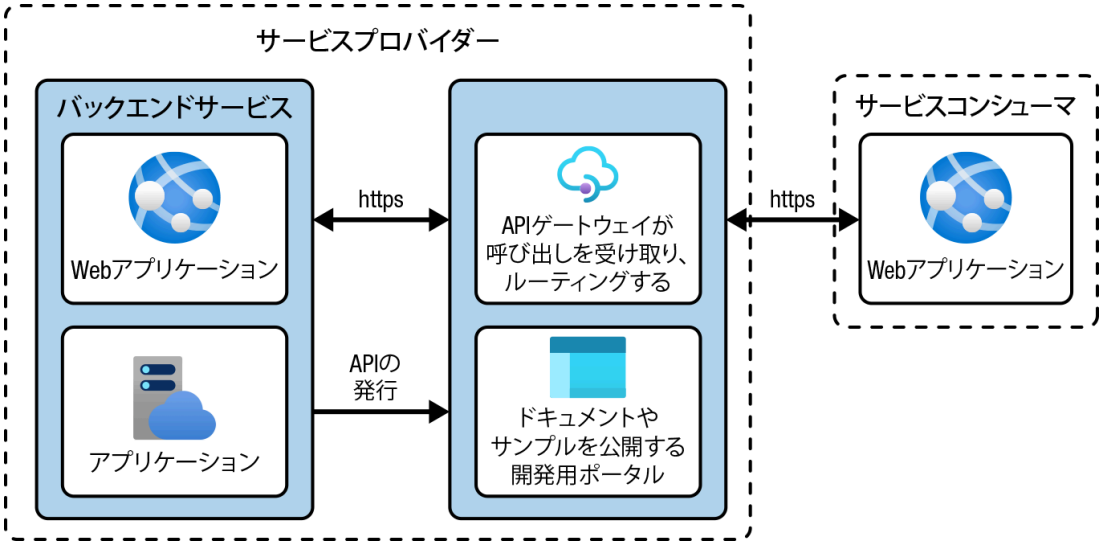

図5-1　APIの作成と利用に関する考え方のまとめ

アプリケーションを直接呼び出す代わりに、SOAではAPI管理サービスによってアプリケーションが統合管理されます。この例では、API呼び出しはAPIゲートウェイを経由します。APIインタフェースは一般的にさまざまな構造を持ち、内部アプリケーションの複雑さを隠蔽します。そのため、データは変換されます。

SOAの用語

SOAにおけるサービスプロバイダーとサービスコンシューマの役割は、ここまで取り上げてきたデータプロバイダーとデータコンシューマの役割に似ています。

- **サービスプロバイダー**とは、サービスを提供するアプリケーションのことです。このアプリケーションは、機能やデータを公開します。

- **サービスコンシューマ**は、サービスを消費するアプリケーションです。要求されたデータは、このアプリケーションに転送されます。

説明を簡単にするために、本章では**サービスプロバイダー**と**サービスコンシューマ**という言葉を使います。ただし、以降の章では**データプロバイダー**と**データコンシューマ**という言葉に戻します。

SOAの当初の目的は、ITランドスケープをより柔軟にし、アプリケーション間のリアルタイム通信を提供することでした。SOA以前は、ほとんどのエンタープライズアプリケーションの通信は、リモートプロシージャコール（RPC）などのクライアントとサーバー間の直接呼び出しによって行われていました[†3]。

[†3] RPCは、クライアントサーバーモデルに似ています。あるプログラムが、ネットワーク上の別のシステムにある別のプログラムに対して、アプリケーション機能を要求するパターンです。RPCでは、メッセージではなく、IDL（interface description language、インタフェース記述言語）を用いて、クライアントとサーバー間の通信を行います。IDLインタプリタは、クライアントとサーバーの両方で実装する必要があります。

アプリケーション間で直接通信を行うデメリットは、アプリケーションの実行環境と密結合してしまうことです。例えば、RPCメソッドを介して通信を行いたいアプリケーションは、同一のクライアントサーバーライブラリとプロトコルが必要になります。これでは、すべての言語ですべてのライブラリやプロトコルが利用できるとは限らないため、アプリケーションの多様性が実現できません。また、互換性の問題もあります。バージョン管理も難しくなります。あるアプリケーションのメソッドが変更された場合、相手方のアプリケーションもすぐに変更しなければなりません。変更しないとシステムが壊れてしまうからです。さらに、スケーラビリティの問題もあります。アプリケーションの数が増えれば、インタフェースの数はすぐに管理できなくなります。一般的には、アプリケーション間のポイントツーポイントのインタフェースを構築する際にも同じ問題が発生します。

これらの問題を解決するために、エンジニアやアーキテクトたちは、新しいソフトウェアアーキテクチャの開発に取り組み始めました。この新しいアーキテ

クチャは、より高いレベルの抽象化と、アプリケーションフロントエンドや、サービス、サービスリポジトリ、サービスバスなどのコンセプトに基づいています。このモデルでは、アプリケーションとそのアプリケーション固有のプロトコルは隠蔽されます。HTTPなどの標準的なWebプロトコルを使ったサービスを介して機能やデータを公開します。アプリケーション間の通信や統合は、サービスバスで処理されます。このような（エンタープライズ）アプリケーション間の通信に対応したテクノロジーや方法論の実装は、**エンタープライズアプリケーション統合**（EAI、enterprise application integration）と呼ばれています。

5.2.1　エンタープライズアプリケーション統合

2000年代初頭、Webの利用が広まるにつれて、SOAの一部であるエンタープライズアプリケーション統合プラットフォームとして、**エンタープライズサービスバス**（ESB）が新たに登場し、さまざまなアプリケーション間やソフトウェアパッケージの結合を処理するようになりました。ESBは、従来のメッセージ指向ミドルウェア（MOM）[†4]やクライアントサーバーモデルを超えて、次のような新しい機能を追加しました。

[†4] **ミドルウェア**または**メッセージ指向ミドルウェア**（https://oreil.ly/Bli92）は、分散システム間のメッセージの送受信をサポートするテクノロジーや製品の総称です。ESBはミドルウェアと考えることができます。ESBはメッセージをルーティングし、これらのメッセージを確実に処理できるようにサポートします。

サービスのオーケストレーション
複数のサービスを、1つの高次の複合サービスにまとめます。

ルールベースのルーティング
指定された条件に基づいてメッセージを受信・配信します。

標準メッセージフォーマットへの変換
ある特定のデータフォーマットから、別のフォーマットへの変換を行います（SOAP/XMLからJSONなど、データ交換フォーマットの変換も含まれます）。

柔軟性のための調停
同一のインタフェースに対して複数のバージョンをサポートし、互換性を確保します。

接続性への対応

バックエンドアプリケーションとの通信や、アプリケーションフォーマットからバスフォーマットへのデータ変換など、さまざまなシステムをサポートします。

ESBは、バスやアダプターを使用して、より迅速にサービスを提供することができるため、アジリティと柔軟性を提供してくれます。従来型のアプリケーションを、わかりやすいインタフェースを持つ最新型のアプリケーションに、ESBは変えてくれます。また、ESBは、抽象化を行い、調停を行うことで、アプリケーション間を分離します。アプリケーションとESBを接続するのがアダプターです。アダプターは、アプリケーションとESBの間のプロトコル変換も行います。

ESBの導入を進めるために、多くのソフトウェアベンダーは、アプリケーションの設計を改めて、アプリケーションやプラットフォームをSOAに対応しはじめました。W3C（World Wide Web Consortium）標準に基づいたサービスインタフェースも登場しました。例えば、XML（Extensible Markup Language）をデフォルトのメッセージフォーマットとするSOAP（Simple Object Access Protocol）などです。

WSDL、XSD、サービスリポジトリとは

XMLベースの通信では、一般的に、WSDLや、XSD、サービスリポジトリが使われます。Webサービス記述言語（WSDL、Web Services Description Language）は、Webサービスとその操作を記述するための標準規格です。XMLで記述されています。WSDLでは、メソッドや、ロケーション、データ型、トランスポートプロトコルを指定します。WSDLが提供されていれば、開発者はWebサービスを開発できるので、統合やソフトウェア開発の作業が容易になります。

WSDLドキュメントは、また別のXMLベースの文書と関連付けられています。XSD（XML Schema Definition）です。XSDドキュメントには、スキーマが記述されています。このスキーマは、XMLドキュメントがどのように構成されているのかを定義するものです。あるXMLドキュメントがスキーマで定められたルールに則しているかどうかを確認するために使用されます。

これらのXMLドキュメントは、一般的にサービスリポジトリに格納されます。通常、これはXMLベースのレジストリです。どのようなWebサービスが利

用可能で、アクセス可能であるかを公開します。サービスリポジトリには、通常、バージョンやドキュメント、ポリシーなども管理されます。

SOAPが普及したのは、ネットワーク通信にHTTP（Hypertext Transfer Protocol）を使用しており、Webサイトの構築に使用されるHTML（HyperText Markup Language）と同じような構造を持っているからです。HTMLとHTTPの規格は、今日のインターネット通信の基礎となっており、現在も使われつづけています。これらの通信規格に基づいて、ESBは変換と統合のブローカーとして機能し、サービスプロバイダーとサービスコンシューマの間に普遍的な接続性を提供します。

SOAの中のESBは、複雑なモノリスになる可能性があります。このことについては、「5.2.6　エンタープライズデータウェアハウスアーキテクチャとの類似性」で、エンタープライズデータウェアハウジングとの比較の際に言及します。

ESBは、メッセージフォーマットを変換し、アプリケーションに最新の通信方法を提供するだけではありません。より複雑な連携が必要な場合には、さまざまな通信形態を提供します。そのうちの2つが、サービスオーケストレーションとサービスコレオグラフィです。これらは、SOAをより複雑で難しいものにするものなので、項をあらためて詳しく説明します。

5.2.2　サービスオーケストレーション

SOAでは、効率性、保守性、調整のために複数のサービスが統合され、サービスプロバイダーとサービスコンシューマの役割が曖昧になることがあります。統合は複雑なことが多く、さまざまな方法で行われます。まず取り上げるのは、複数のサービスを組み合わせてラップし、1つの大きなサービスとして公開する、**サービスアグリゲーション**です。この方法は、管理の手間を削減し、（1つまたは複数の）サービスコンシューマのニーズを満たしたいときに役立ちます。この方法でのアグリゲーターは、さまざまなサービスの調整や、呼び出し、結合に責任を持ちます。

サービスアグリゲーションには、**サービスオーケストレーション**が必要です。サービスオーケストレーションとは、さまざまなサービスを調整して組み合わせ統合することです[†5]。この作業は、一般的に、データの結合、分割、変換、検査、破棄などの技術的な理由で行われます。サービス間で必要となるサービスフローが単純な場合、サービスオーケストレーションプロセスは短い時間で実行され、比較的単純な組み合わせで構成されます[†6]。例えば、ある顧客の契約内容をすべて取得する際に、顧客の製品サービスという別のサービスをまず呼び出す場合などです。また、サービスを組み合わせて、意思決定ロジックや、別の実行パスを必要とするような複雑なフローを長時間実行することもできます。サービスオーケストレーションは継続時間についてあまりはっきりと定めていませんが、主に長時間動作するサービスの場合には、サービスオーケストレーションという言葉が使われます。サービスオーケストレーションは、さまざまな理由で行われる可能性があるため、ある程度言葉を使い分けるようにしてください。

[†5] **サービスコンポジション**という言葉を使う人もいます。サービスコンポーザビリティの原則では、複数のソリューションで容易に再利用できるようにサービスを設計することが推奨されています。つまり、既存のサービスを利用して新しいサービスを作れるようにするということです。サービスコンポジションは、サービスが結合されていることを示す言葉ですが、その方法については言及していません。

[†6] **サービスフロー**は**マイクロフロー**とも呼ばれます。一般的に、1つのスレッドと1つのトランザクションで、中断されることなく実行されます。実行期間は短く、通常、同期的サービスのみを使います。プロセスの状態は永続化されないのが普通です。

オーケストレーションはESBで行うべきか、消費側で行うべきか

サービスを組み合わせて統合するには、2つの方法があります。1つ目は、ESBのアグリゲーションサービスを利用することです。このサービスを使えば、サービスコンシューマは1回の呼び出しですべてのデータを取得することができます。ESBがオーケストレーターの役割を果たし、それに付随するすべての作

業を行ってくれます。ただし、オーケストレーションを行うESBは、結果的に単一障害点となります。

もう一つの方法は、消費アプリケーションにオーケストレーターの役割を担わせることです。この場合、消費アプリケーションは呼び出し順序を管理し、結果を結合（統合）しなければなりません。

どちらの方法にも長所と短所があります。ESBにオーケストレーションを任せることの利点は、オーケストレーションやビジネスロジックを再利用しやすくなり、データ消費が容易になることです。欠点は、すべてのロジックを組み合わせるのに十分な性能をESBが提供しなければならないことです。さらに大きな欠点は、サービスアグリゲーションがさまざまなサービスのファサードとして機能するため、サービスプロバイダーの役割がはっきりとしなくなることです。アグリゲートされたサービスは、一般的に、オーナーが異なるさまざまなサービスを組み合わせたものになります。このため、アグリゲーションサービスの完全性を保証するオーナーはどこにもいません。

消費アプリケーションにオーケストレーションさせることは、柔軟性という利点があります。ただし、すべてのコンシューマにオーケストレーションを実装させる必要があるため、効率的ではありません。

ビジネスプロセス管理（BPM、business process management）という理由で、オーケストレーションが行われることもあります。この場合のプロセスにはビジネスロジックが含まれています。また、一般的に長時間実行されます。一連のステップを管理し調整するには、各ステップのプロセスステータス（状態）を保存する必要があるので、データベース中に永続化されます。長時間実行されるビジネスプロセスは中断可能です。また、ほかのプロセス（または人間のタスク）が終了するのを待つことができます。これらのプロセスは一般的に非同期であるため、ワークフローや長時間実行するプロセス管理が複雑になります。この複雑さに対処するため、視覚的で直感的なBPMソフトウェアを使って行うのが普通です。

BPMソフトウェアのプロセスレイヤーは、アプリケーションとサービス間のデータ交換フローについて、すべての情報を保持します。順序や、依存関係、ステータス、タスク、ルール、待ち時間などです。このようなプロセスレイヤーに、アプリケーションデータを保存しては**なりません**。BPMソフトウェアはアプリケーションデータを管理するためのものではありません。BPM APIを使う目的は、プロセスを開始し、ワークフローのステータスに関する情報を取得するためです。顧客や、製品、契約に関するデータを保存するために使用しないでください。

BPMを使うのは、一般的に、さまざまなドメインのさまざまなアプリケーション間で、（長時間実行する）プロセスをオーケストレーションする必要がある場合です。BPMを使えば、ドメインはビジネスプロセスをビジネスアプリケーションロジックから分離できます。ただし、アプリケーション境界内でプロセスやデータをオーケストレーションする場合は、アプリケーション自体で個別に行うのが良いでしょう。

BPMが長時間実行するプロセスを管理する場合、処理は一般的に非同期になります。BPMはすべての依存関係を管理し、各プロセスの個々の状態を記録し、いつトリガーをかけ、データを取得し、保存するかを把握しています。プロセスを起動したり呼び出したりするために使われるAPIは、**プロセスAPI**と呼ばれることもあります。

サービスオーケストレーションは、ほかのタイプのオーケストレーションと誤解されることがよくあります。サービス指向アーキテクチャのコンテキストでは、オーケストレーションとは、エンドツーエンドのサービスを提供するために必要となる、運用プロセスや機能プロセスを呼び出して実行することです。一般的に、この形式のオーケストレーションは、アプリケーションAPIを使用し、BPMソフトウェアによって管理されます。

ほかの形式のオーケストレーションとしては、スケジューリングや、自動設定、調整などがあります。以下に例を挙げます。

- オペレーティングシステムにおけるプロセスのスケジューリング（例：Unix系システムのcronなど）。
- アプリケーション境界内のプロセスやタスクスケジューリング（例：内部/ネイティブのアプリケーションスケジューラなど）。
- データ処理を自動化するためのデータオーケストレーション（例：データ移動や、データプレパレーション、ETLプロセスなど）。Apache Airflowは、この種の処理のためによく使われるツールです。
- インフラストラクチャーのスケジューリングとオーケストレーション（例：VMのプロビジョニングや、システムのシャットダウンなど）。

このほかのオーケストレーションの形式としては、継続的インテグレーションと継続的デリバリー（CI/CD）があります。ソフトウェアの監視や、構築、テスト、リリース、統合を自動化するプロセスです。この形式のオーケストレーションでは、ほかの形式のオーケストレーションを使用したり組み合わせたりするのが一般的です。

5.2.3　サービスコレオグラフィ

サービスプロバイダーとサービスコンシューマの役割を曖昧にするパターンとして、**サービスコレオグラフィ**もあります。サービスコレオグラフィとは、プロセス間のやり取りとビジネスロジックを、独立したパーティに分散させた上で、何らかのビジネス上の目的を達成するために協調する方法を指します。サービスコレオグラフィでは、意思決定ロジックが分散され、各サービスが環境を見て、自律的に行動します。BPMのような中央集権的なオーケストレーターはなく、どのパーティもほかのパーティのプロセスを完全にコントロールすることはできません。各パーティが、プロセスを部分的に所有し、その部分について責任を持ちます。サービスコレオグラフィでは、図5-2に示すように、さまざまなサービスプロバイダーとサービスコンシューマの間で、リクエストがピンポンのように行き来します。サービス間のやり取りを制御するロジックは、中央プラットフォームの外側に置かれます。サービス自身で、次にどのサービスをどのような順番で呼び出すかを決定します。

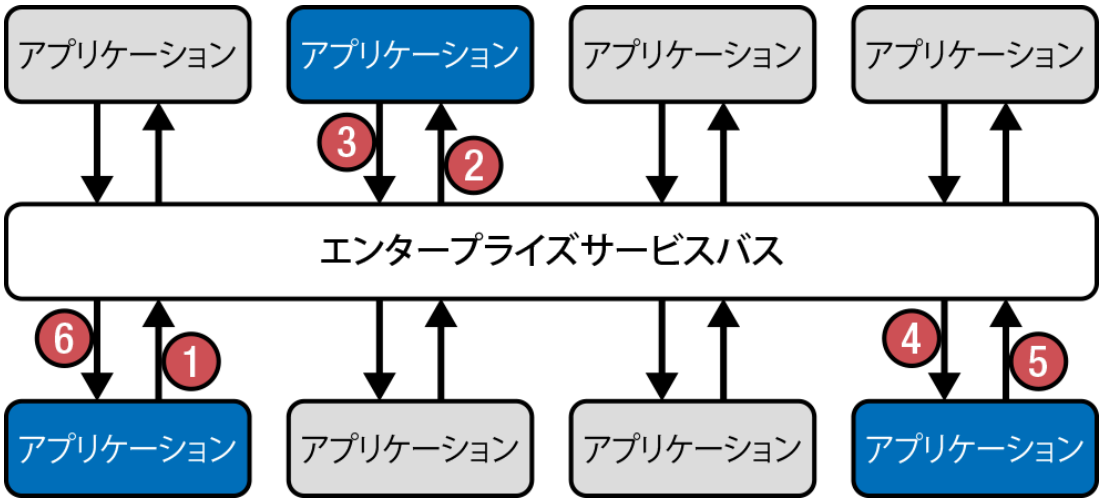

図5-2　サービスコレオグラフィは、サービス実行とコミュニケーションを非中央集権的に実現するアプローチ

サービスコレオグラフィや、アグリゲーション、オーケストレーションを組み合わせることもできます。いくつかのサービスやプロセスを中央で管理し、この中央管理しているものとは別に、アグリゲーションサービスからほかのサービスのワークフローを起動したりすることができます。どのような状況においても、サービスプロバイダーとサービスコンシューマの役割は曖昧になります。ドメイン全体の完全性を保証できるドメインが存在しないからです。

Martin Fowler氏は、“Event Collaboration”（https://oreil.ly/ajRx-）という投稿の中で、ほかのアプリケーションからの情報を元に、通信を行うアプリケーションという面白いパターンを提案しています。データが必要なときにリクエストを行う代わりに、アプリケーションは物事が変化したときにイベントを発生させます。ほかのアプリケーションはこれらのイベントを受け取り、適切に反応します。これは、6章で説明するイベント駆動型アーキテクチャに非常によく似ています。

これまでのいくつかの項では、主にプロセス間のやり取りとアグリゲーションという観点から、サービスを見てきました。これから項を変えて、サービスが

動作するコンテキストという観点で、サービスを分類してみたいと思います。

5.2.4　パブリックサービスとプライベートサービス

サービスは多くの異なる目的を対象にしています。そのため、多くの人々がSOAサービスを、パブリックとプライベートなどの種類に分類しています[†7]。**パブリックサービス**は、**ビジネスサービス**と呼ばれることもあり、ある特定のビジネス機能を提供します。これらのサービスは、多くの場合、データやロジックをグループ化し、ビジネスにとって意味のある方法で表現します。そのほかのサービスは、**プロセスサービス**と呼ばれることもありますが、プロセスを開始したり、ビジネスワークフローを起動したりなど、動作面の面倒を見るサービスもあります。ビジネスサービスは抽象的である場合が多く、ある特定のビジネス機能について権限を持ち、アプリケーション境界を越えて動作します。

[†7] 例えば、OpenAPIHub（https://oreil.ly/xiHZ2）では、Public API、Private API、Partner APIという分類を使用しています。

プライベートサービス（**テクニカルサービス**）とは、必ずしもビジネス機能を表すわけではないサービスです。内部ロジックの一部として使用され、データベースのテーブルを公開するなど、技術的なデータやインフラストラクチャー機能を提供します。プライベートサービスがパブリックサービスの入力になることもあります。このような場合、最終的には、それらのサービスはビジネス機能を提供する別のサービスにラップされる場合が多いです。

インフラストラクチャーサービスや**プラットフォームサービス**という言葉を使う人もいます。これはインフラストラクチャーに対する関心事をドメインから抽象化したサービスです。これらのサービスは、通常、組織全体で標準化されており、認証や、認可、監視、ログ、デバッグ、監査など、すべてのドメインに適用される機能が含まれます。

一般にプライベートサービスは、ビジネス上の問題とは無関係で、あまり抽象化されないため、密結合になりがちです。このようなサービスは、幅広く利用されることは想定されておらず、アプリケーション境界内でのみ動作するため、中央リポジトリで公開するべきではないという意見や、ドメイン間で共有すべきではないという意見もあります。

5.2.5　サービスモデルとカノニカルデータモデル

さまざまなアプリケーションを接続する際、エンジニアや開発者は、サービスが提供するデータモデルやビジネス機能について共通の認識を持つ必要があります。これが、サービスモデルとカノニカルデータモデルの役割です。**サービスモデル**とは、サービスのインタフェースがどのようになっているか、データとそのエンティティや、関係、属性がどのようにモデル化されているかを記述したものです。また、定義や、ほかのサービスとの依存関係、バージョン番号、構文、プロトコル、アプリケーションの所有者や稼働状況などの情報が含まれる場合もあります。

サービスモデルの目的は、サービスの利用や統合を容易にするための情報を提供することです。通常、プラットフォームには依存しないので、システムについては記述しません。サービスモデルは、さまざまなスコープで使用され、さまざまな形式で存在します。ドキュメントとしてのみ存在し、使用されることの多いコンテキストを記述しているものもあります。コードとしてツールに組み込まれ、データとそのすべての属性およびプロトコルを記述しているものもあります。

カノニカルデータモデルは、サービスモデルとは異なり、サービスや言語を標準的かつ統一的に定義することを目的としています。サービスモデルがアプリケーションに近いものであるのに対し、カノニカルデータモデルはさまざまなサービスモデルの整合性や標準化のために使用されます。グローバルなスキーマを用いて、すべてのサービスを統一したいと考える人もいます。そのため、カノニカルモデルは大規模で複雑なものになりがちです。

5.2.6　エンタープライズデータウェアハウスアーキテクチャとの類似性

ESB、アグリゲーション、サービスのレイヤー化、カノニカルデータモデルの組み合わせにより、SOA実装は必要以上に複雑化しています。実際、エンタープライズデータウェアハウスアーキテクチャと同じような課題に、多く直面しています。

5.2.6.1　カノニカルモデルの大きさ

最初の類似点は、カノニカルモデルが使用される規模です。多くの組織では、単一のエンタープライズカノニカルモデルを使用して、**すべての**サービスを記述しようとします。エンタープライズカノニカルモデルを使用するには、関係するすべてのチームやパーティ間での調整が必要になります。この方法では、サービスには大量のオプション属性が付いてしまいます。すべての関係者が自分たちの要件を反映するように求めるからです。その結果、すべての人のニーズを満たすために妥協した、誰にもうれしくないモンスターインタフェースモデルができあがります。あまりにも汎用的なものになるので、サービスが正確に何を提供しているのか、何をしているのか、誰にもわかりません。同様の問題はエンタープライズデータウェアハウスでも見られます。この点については、「1.9.1　エンタープライズデータウェアハウス：信頼できる唯一の情報源」で説明しました。

5.2.6.2　レガシーミドルウェアのラッパーとしてのESB

次に似ている点は、大規模なレガシーミドルウェアやESBでよく見られる、サービスのレイヤー化とアグリゲーションについてです。これらのアーキテクチャの多くは現在も存在し、メッセージのルーティングや、統合、変換、プロセスオーケストレーションの大部分を行っており、これらのプロセスの状態の管理と永続化も行っています。一部の企業では、これらのレガシーミドルウェア

システムを最新のテクノロジーでカプセル化しています。このようにすれば、複雑さが抽象化され、RPCの代わりにJSONを使用するなど、最新の通信方法が使えるようになります。しかしながら、古いミドルウェアを新しい技術でカプセル化して提供していくと、多層化（図5-3）が進み、複雑さが増します。これは、たとえ小さな変更を加えただけであっても、連鎖的に変更を行わなければならなくなるリスクがあることを意味します。

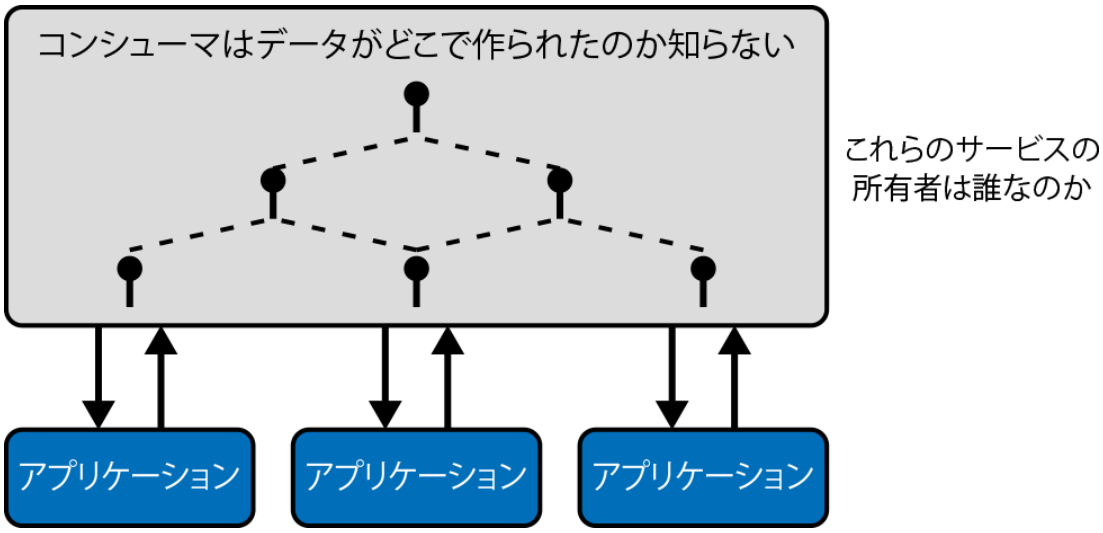

図5-3　ほかのサービスの上にサービスを重ねることは透明性を低下させる

気づかないうちに、サービス呼び出しの複雑なフローができあがってしまいます。データがどこから来ているのか、どのようなほかのサービスやプロセスが動いているのか、正確に把握できないままサービスが呼び出されます。そして、多くの場合、誰もこれらのサービスについてオーナーシップを持とうとしません。BPMでも同じことがよく起こります。テクニカルオーケストレーションとプロセスオーケストレーションが混在していたり、すべてのドメインロジックとプロセスロジックが中央集権的にまとめられていたりするため、どのプロセスがどのような関係にあるのかを把握するのが難しくなります。

5.2.6.3　アプリケーションの状態を管理するESB

最後にお伝えする類似点は、ESBが複数のアプリケーションの状態を保存する場所になっていることです[†8]。アプリケーションやビジネスプロセス管理機能の中で状態を管理するのではなく、ESBを使うことで、長期間実行するビジネスプロセスをオーケストレーションしたり、さらにひどいことに、アプリケーションが保持すべき状態を永続化させたりします。この結果、ESBはデータベースになります。この方法には、密結合のリスクがあります。何らかの理由でESBが停止した場合、状態が失われ、すべてのプロセスが自身の状態を失う

ことになります。また、アプリケーションが変更された場合にもリスクがあります。アプリケーションに関する状態にも変更が必要となり、すべてのアプリケーションがESBに永続化された状態に依存している場合、アプリケーション間の調整を慎重に行って、ESB中の状態を安全に変更しなければなりません。このように、ESBについては、一時的なセッション管理とキャッシュ以外は、ステートレスを原則とすべきです。

[†8] **状態管理**とは、ユーザーの入力やプロセスの状態など、アプリケーションの状態を管理することです。

5.3　API管理に対する最新の考え方

ESB統合プラットフォームは、SOAを必要以上に複雑にしました。企業はESBの「E」を文字通り受け止め、モノリスを構築して、すべてのサービス統合の面倒を見るようにしました。中央チームが、サービスの再利用性と設計がどうあるべきかを決定したのです。その結果、相互コミュニケーションが頻繁となり、イノベーションとチームのアジリティが損なわれました。

一方で、マイクロサービスの台頭により、アプリケーションのモジュール化や設計が変化しており、今日では、最新型のサービスを使用すれば、信じられないほど高速かつ容易にAPIを構築できるようになっています。デジタルエコシステムを考えると、クラウドや、SaaS、APIエコノミーにより、APIは企業の枠を超えて広がっています。最新のデータベースやアプリケーションは、デフォルトでRESTfulAPIをサポートしています。当然のことながら、SOAに対する新しい考え方が必要になります。

5.3.1　連合型責任モデル

このようなトレンドに基づき、中央集権的なESBベースのアーキテクチャを破壊しなければならないとしたら、新しいAPIアーキテクチャはどのようになるべきでしょうか。中央集権的なモデルではなく、サービスを構築し公開する責任は、その基盤となるドメインに委ねられます。データプロダクト管理と同じように、連合モデルにしたほうが良いのです。このようにすれば、ドメインは、ほかのドメインを邪魔することなく、それぞれのスピードで進化することができます。

この連合型運用モデルでは、ビジネスロジック、プロセスの実行、データの永続化、コンテキストの検証などの責任もドメインに戻されます。ESBを（中央集権的な）統合プラットフォームとして使用することは避けるべきです。集約とオーケストレーションは、ドメインの境界内でのみ許されます。連合型モデルはAPIの設計にも影響します。データプロダクトの作成と同じく、ユーザーフレンドリーな設計原則がここでも当てはまります。また、APIは、データプロダクトと同様に、ドメイン言語を使って開発されなければなりません。

API管理については、以下のような原則があります。

- 複雑なシステムではなく、RESTリソースモデル（次のコラムで説明）を使用してビジネス機能やデータを公開します。これには多くの作業が必要です。ビジネスコンセプトやそのプロパティを理解すること、関係に適切に名前をつけること、それに応じてスキーマを設計すること、適切なIDと一意性の確保などに専念してください。
- できるだけデータに近いところで最適化し、APIレイヤーでのオーケストレーションの負荷が重くならないようにします。例えば、アプリケーションとAPIがオンプレミスでホストされている場合は、セルフホスト型のAPIゲートウェイインスタンスを使用します[†9]。
- ドメインロジックはそのドメイン内に保持します。
- 適切なレベルの粒度でコンシューマ向けサービスを構築してください[†10]。
- シンプルにして、最新のコミュニティ標準を使用してください。
- サービスオーケストレーション（複数のAPIエンドポイントを1つにまとめること）は、ドメインの境界（コンテキスト境界）内でのみ行うようにしてください。
- 一貫性のあるドメイン識別子を使って、ドメインがリソース間の相互接続を識別できるようにしてください。

[†9] Azure API Managementドキュメント（https://oreil.ly/XdCJo）では、セルフホストインスタンスを管理する上で考慮すべき点を概説しています。

[†10] Filipe Ximenes氏とFlávio Juvenal氏は、正しくフォーマットされ、リンクされ、バージョン管理されたリソースをRESTful APIがどのように表現す

るかについて、ブログ（https://oreil.ly/sEEyW）にまとめています。

リソース指向アーキテクチャ

ソフトウェアエンジニアリングの世界には、**リソース指向アーキテクチャ**（ROA、resource-oriented architecture）と呼ばれるパターンがあります（https://oreil.ly/N2L88）。ROAは、RESTfulなアーキテクチャに対する具体的なガイドラインです。データは、同じ領域のコレクション、つまりリソースとしてモデル化されます。論理的に1つにまとめるべきものは、リソース（コレクション）としてまとめられます。各リソースは、URIで識別できるエンティティになります。例えば、リソース**customers**は、名前や、住所、連絡先など、顧客に関する情報を返すために使用されます。このパターンのデータリソースには、名詞をつけるのが一般的です。名前を使ってリソースを検索できるようにして、その名前でどのようなリソースなのかがわかるようにしなければなりません（対照的に、SOAは一般的に動詞を使います）。

これらのガイドラインに従うことで、APIアーキテクチャをスケーラブルにすることができます。ドメインは分離されたままで、作業を何度も繰り返すことなく、ほかのサービスを利用することができます。このような方法を推進するには、より軽量な統合サービスを検討する必要があります。

5.3.2　APIゲートウェイ

RESTful APIやアプリケーションの近代化の流れを受けて、アプリケーション間の通信やサービス統合が変化し始めました。REST（https://oreil.ly/9RKWZ）は、本章のコラム「リソースと操作」で紹介した、ステートレスな通信を実現するアーキテクチャパターンです。最新のWebベースアプリケーションで使われています。RESTは、シンプルであることを基本としています。リソース（関連情報を表現したチャンク）は、URI（Uniform Resource Identifier）で識別されます。そして、ハイパーメディアリンクで接続することができます[†11]。また、HTTPを使ってインタフェースメソッドも統一しています。RESTful APIは、その相互運用性と柔軟性により、Webサイトや、モバイルアプリ、ゲームなどでよく利用されるようになっています。

[†11] HATEOAS（Hypermedia as the Engine of Application State）が、RESTリソースのリンクに使用されます。RESTful API Tutorial Webサイトに、HATEOASのチュートリアルがあります（https://oreil.ly/BnfAg）。

RESTful APIでは、CRUD（create、read、update、delete）がよく使われます。CRUDは、データベースやデータリポジトリで実行される基本操作に対応しています。データベースの操作では、レコードやデータオブジェクトを直接扱うのが一般的です。一方で、**REST**は、リソースの表すものを操作します。各リソースはURLで識別されます。これらのリソースは、一般的に、データオブジェクトではなく、複雑で高レベルに抽象化されたものです。

例えば、顧客との契約はリソースになります。つまり、リソースには、「契約」テーブルのレコードだけでなく、「顧客」リソースとの関係や、その契約が添付されている「合意書」、その契約関連のほかのコンテンツも含まれます。

RESTful APIが普及するにつれ、メッセージ形式やプロトコルも変化し始めています。SOAPプロトコルは、比較的大きなXMLメッセージ形式を使っていました。これが、構文が小さく軽量で高速なJSON（JavaScript Object Notation）に変わりました。また、JSONメッセージは、簡単にキャッシュしたり、キーバリューデータベースに保存できたりするというメリットもあります（APIや統合プラットフォームにおいて、以前にリクエストされたデータを保存しておき、必要に応じてそのデータを提供することができます）。

RESTful APIは、レイヤーや中間層を追加することで、コンシューマに対して難読化することができます。つまり、サービスコンシューマにレイヤー内での構成を明らかにすることなく、処理を実行することができます。これは、一般的に、スケーラビリティ（キャッシュやロードバランシング）やセキュリティを強化するために必要となるものです。

最新のアプリケーションや、プロトコルとメッセージ設計の変化も、APIアーキテクチャに影響を与え始めています。より軽量な統合コンポーネントとして、**APIゲートウェイ**が登場しました。APIゲートウェイは、アダプターのオーバーヘッドやESBの複雑な統合機能を持ちません。それでも、カプセル化を可能にし、APIの利用状況をコントロール、保護、管理、レポートする管理機能を提供します。APIゲートウェイは、一般的に、API管理ソフトウェアの一部となり、コンポーネントを2つ持ちます。1つは、ドキュメント、コードサンプル、利用可能なAPIのリストを保持する開発者ポータルです。もう1つは、HTTP呼び出しを受け付け、基盤となるサービスやアプリケーションにルーティングするためのAPIゲートウェイコンポーネントです。APIゲートウェイ自体の配備については、さまざまな方法を用いることができます。小規模な組織では通常、データセンターのエッジまたはクラウドに単一のAPIゲートウェイインスタンスを配備します。大規模な組織では、さまざまな場所（ドメインチーム、組織部門、各地域内）に複数のインスタンスを配備するのが一般的です。

5.3.3　プロダクトとしてのAPI

新しい分離パターンを理解したところで、ESBの役割を批判的に見ることができるようになりました。ESBが必要になるのは、重い処理を行う場合だけです。ほかのすべてのユースケースでは、APIゲートウェイの方を使うべきです。結論として、ESBは、レガシーシステムのモダナイゼーションと、非常に負荷の重い技術的な統合の問題を解決するためだけに使ってください。中央統合レイヤーの使用を控えることで、ドメインはアプリケーションや、サービスとデータが公開される方法をより真剣に検討する必要があります。ドメイン

は、APIを自らのプロダクトとしてとらえ、管理することが求められます。これには、次の項で紹介する事項を検討しなければなりません。

5.3.4　複合サービス

ドメイン境界内でプロダクトとしてAPIを管理する場合、複合サービスや集約サービスの設計に特に注意する必要があります[†12]。**図5-4**に示すように、複合APIは、一般的に、1回の呼び出しで一連のREST APIリクエストを実行するAPIのことです。複合APIが求められるのは、例えば、コンシューマが2つのシステム（例えば、顧客と注文）のデータを組み合わせたものを必要とした場合などです。集約する複合サービスを構築する場合、統合ロジックと変換ロジックを可能な限りコンシューマに任せるようにします。こうすることで、コンシューマ固有のビジネスロジックや統合ロジックを、コンシューマドメインが所有することになります。例えば、複合サービスに複雑なビジネスロジックが含まれる場合、ドメイン境界内にコンポーネントやマイクロサービスを新たに配備することで対応します。つまり、ビジネスロジックと対応するコンテキストを所有するパーティが、コンポーネントと対応するAPIを所有しなければなりません。このような協調モデルを採用した場合、API呼び出しのルーティングフローの中で、APIゲートウェイの果たす役割は小さなものになります。リクエストと発信元が有効かどうかを検証するだけです。

[†12] Azureのドキュメント（https://oreil.ly/qvafu）には、Luaを用いたゲートウェイ集約NGINXサービスという、具体的でシンプルな例が記載されています。

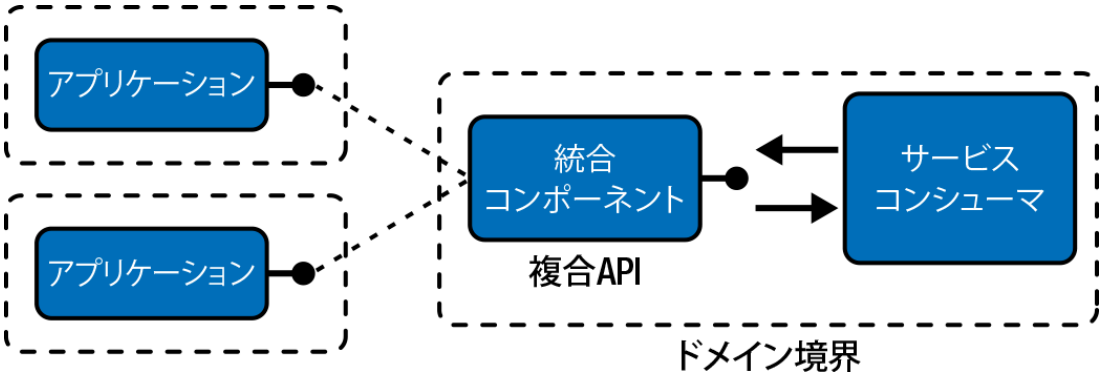

図5-4　複合サービスでは、統合ロジックと変換ロジックはコンシューマ側に委ねられる

複数のドメインが複合サービスに関与する場合、一貫性、完全性、セキュリティに細心の注意を払う必要があります。基盤となるサービスをそれぞれ中央のリポジトリに登録し、ドメインがほかのドメインとの依存関係を把握できるようにすることが重要です。また、ほかの複合サービスの上に複合サービスを構築することはお勧めしません。なぜなら、APIからAPIへの呼び出しでオーケストレーションが混沌とした網の目のようになってしまうからです。セキュリティに関しては、ルールと意思決定ロジックをできるだけ元のドメイン内に留めてください。ほかのドメインがルールを上書きしたり、ローカルルールを優先したりしてはなりません。

5.3.5　API契約

複数のチームが同じAPIに関わる場合、すべてのチームが仕様と設計に合意することが非常に重要です。そこで登場するのがAPI契約（サービス契約）です[†13]。

[†13] Azure API Management DevOps Resource Kit（https://oreil.ly/uOK71）は、Azure API ManagementでAPI DevOpsを構築する例を集めたすばらしいリポジトリです。

API契約（サービス契約）とは、APIがどのように設計されているか（構造、プロトコル、バージョン、メソッドなど）を記録したドキュメントです。サービスプロバイダーとサービスコンシューマの間で合意を形成するために利用できます[†14]。API契約の形式としては、**OpenAPI Specification**（https://oreil.ly/cZ9De）（旧称**Swagger Specification**）を使うのが一般的です。この形式のように、契約をコード化することには、人間もシステムも仕様を解釈できるというメリットがあります。また、サービスプロバイダーはサービスコンシューマからテスト手順を受け取ることができるので、変更を行った場合に、サービスが壊れるかどうかを検証できます。

[†14] Martin Fowler氏は、コンシューマ駆動型契約にどのような利点があるかについて深く考察しています（https://oreil.ly/Qcn-3）。

5.3.6　APIディスカバラビリティ

データプロダクトのカタログと同じように、サービスレジストリや開発者ポータルを使ってサービスを把握できるようにしてください。**サービスレジストリ**は、APIという重要な情報を中央のリポジトリに保存するツールです。サービスレジストリにより、サービスの再利用が促進されます。サービスレジストリからは、オーナーシップ、バージョン管理、変更、使用方法などに関する重要な情報が提供されます。

5.4　マイクロサービス

ドメインからサービスを分離することを推進し、複雑で密統合されたサービス指向アーキテクチャを分解したので、これから新たなトレンドであるマイクロサービスについて説明しましょう。2章で触れましたが、マイクロサービスアーキテクチャは、アプリケーションを小さく管理しやすいパーツに分解するアーキテクチャパターンです。**疎結合サービス**という言葉を使う人もいます。また、すぐに説明しますが、SOAと重なる部分があります

5.4.1　機能（Function）

マイクロサービスアーキテクチャを設計する際には、FaaS（Function as a Service）というモデルも使えます。これは、クラウドコンピューティングにおいて比較的新しいサービスです。個々の「機能」（アクション、ビジネスロジックのパーツ）が配備され実行されます。FaaSを利用することで、エンジニアはアプリケーションの機能を開発、実行、管理することができます。一般的に、このプロセスに関連するインフラストラクチャーの構築や保守は不要になります。このモデルでは、使用量に応じた従量制ライセンスの元で、機能がオンデマンドに実行されます。このようなインフラストラクチャーのモデルは、**サーバーレス**とも呼ばれています[†15]。

[†15] **サーバーレス**といっても、サーバーがないわけではありません。サーバーレスは、クラウドコンピューティングの実行モデルです。サーバーレスでは、インフラストラクチャーについての詳細は、ユーザーからまったく見えなくなります。Martin Fowler氏の説明を参照してください（https://oreil.ly/y2PnK）。

各機能には、通常、その機能に対応するAPIがあります。このモデルのAPIは、機能を呼び出したり、起動したりするために必要なものです。これらの機能と、対応するAPIに対する原則は、ほかのAPIとまったく同じです。ドメイン境界内で内部的に使われる場合、機能を直接使ってもかまいません。ドメイン境界を越える場合、サーバーレスAPIは誰かが所有し、契約を行い、APIリポジトリに登録する必要があります。

APIゲートウェイによって機能は隠蔽されますが、イベント駆動型アーキテクチャ（6章）と重なる部分が多くあります。サーバーレスの機能は、多くの場合、非同期なシナリオで使われます。機能は遅くなることがあるためです。クラウドコンピューティングでは、高速なリソースの大部分は、高価な計算能力を使うようなアプリケーションに割り当てられます。そして、残りの「安い」計算資源に、サーバーレス機能が割り当てられます。このため、機能のレスポンスタイムはあまりあてになりません。非同期型の機能では、メッセージキューを使用して結果を保存し、ほかのプロセス（機能）がその結果を取り出すのを待つように設計することができます。

5.4.2　サービスメッシュ

2章では、マイクロサービス間の通信にサービスメッシュを使用することについて説明しました。サービスメッシュとAPIゲートウェイの間には、微妙な違いはあるものの、基本的には同じように動作します。どちらも分離や、監視、検索性、ルーティング、認証、スロットリングを処理します。主な違いは、内部のサービス間通信にあります。サービスメッシュ内の通信は外部にルーティングされず、マイクロサービスが動作する環境のサービス境界内にとどまります。

APIゲートウェイと同じように、サービスメッシュはコントロールプレーンとデータプレーンの2つのサービスを使って実装されます。**コントロールプレーン**はルーティングポリシーやセキュリティなどを設定するサービスです。**データプレーン**はルーティングが実行され、ポリシーやセキュリティが適用されるサービスです。

マイクロサービスアーキテクチャを、APIアーキテクチャ全体に組み入れる方法については、**図5-5**を見てください。この図では、3つの異なるAPIパターンを示しています。

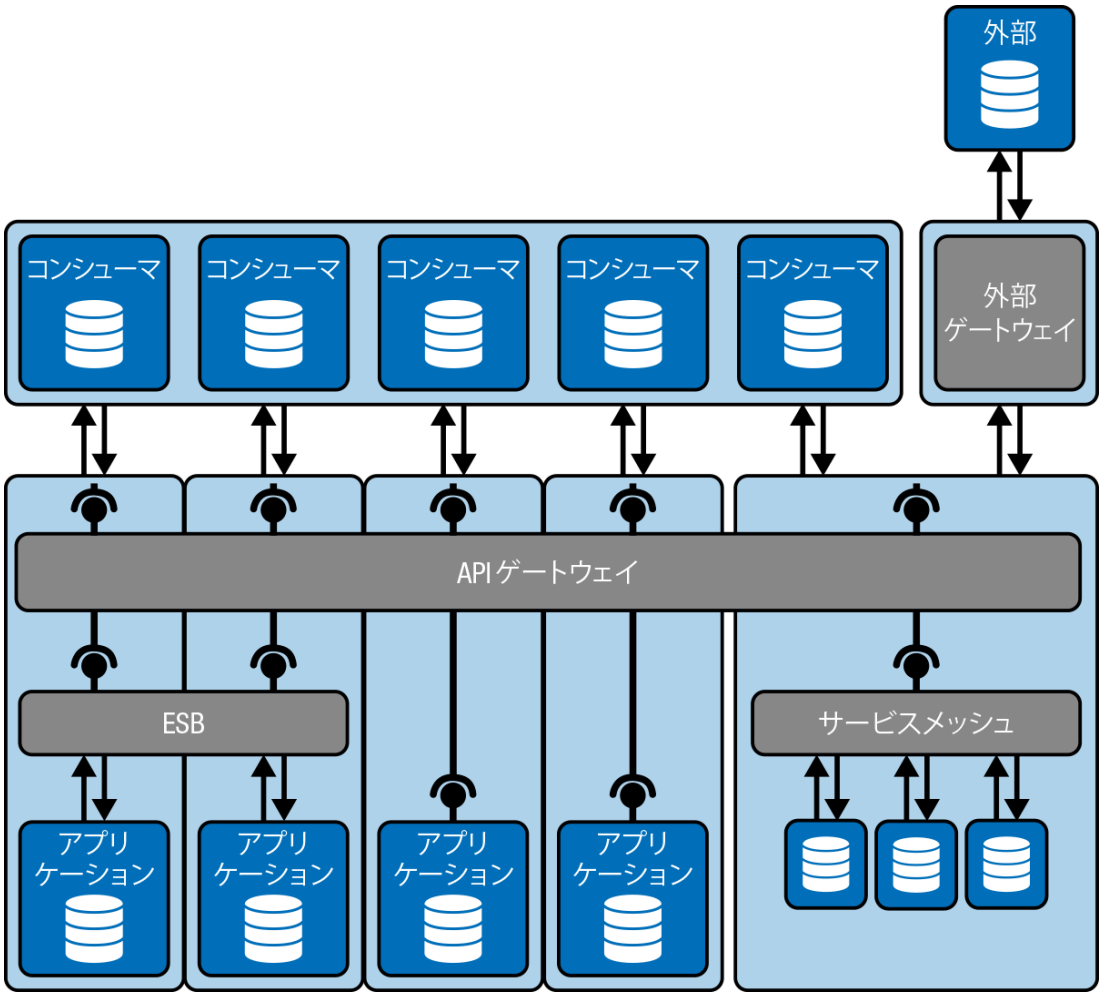

図5-5　APIアーキテクチャはマイクロサービスを否定しない。また、マイクロサービスもAPIアーキテクチャを否定しない

- 中央下は、最新型のアプリケーションです。APIゲートウェイを介して直接データを公開できます。このパターンでは、アプリケーションはREST/JSONなどの最新の通信プロトコルを使用します。
- 左下は、レガシーアプリケーションです。ESBを追加して分離を行う必要があります。ESBサービスはAPIゲートウェイでラップすることができます。
- 右下は、マイクロサービスです。マイクロサービスは内部でサービスメッシュを介して通信します。外部に公開する必要があるAPIは、APIゲートウェイを介して分離します。

サービスメッシュとAPIゲートウェイの両方が常に必要なのか疑問に思うかもしれません。現状では、両者を併用するのがベストプラクティスです。サービスメッシュは、マイクロサービスベースのアーキテクチャにおいて、サービス間通信を管理するインフラストラクチャーレイヤーです。一般的にはマイクロサービスとして実装されます。企業内の分離に使用することもできますが、すべてのドメインがマイクロサービスを使用しているわけではないので、唯一の手段として使うことはお勧めしません。マイクロサービスベースのドメインと

マイクロサービスを使用しないドメインを分離するための方法としては、スタンドアローンコンポーネントとして動作するAPIゲートウェイを配備するのが現実的です。

5.4.3　マイクロサービスドメイン境界

マイクロサービスを管理する際の懸念事項としては、サービスプロバイダーとサービスコンシューマのマイクロサービスの周囲に置かれる論理的境界も挙げられます。多くのマイクロサービスが同じプラットフォーム上で動作している場合、論理的な境界を引くことは重要です。サービスのスコープと分離を活用すれば、ドメインがその複雑さやほかのドメインとの依存関係を管理しやすくなります。ドメイン駆動設計モデルを使って、サービスに境界を設定します。もし、コンテキスト境界が変われば、境界も変更して、サービスを分離しなければなりません。

この論理に従えば、マイクロサービスは2度分離することになります。最初の分離はドメイン境界内で行われます。内部のサービス間通信のレベルです。あるマイクロサービスは同じドメイン内の別のマイクロサービスから分離されます。2つ目の分離は、ドメイン間通信のレベルで行われます。あるマイクロサービスが異なるドメインのマイクロサービスと通信したい場合です。マイクロサービスAPIの設計では、前者のほうが、より技術的なものになるでしょう。これらのAPIは、ほかのドメインに直接共有されることは想定されていないからです。また、サービスカタログで公開されることもありません。後者のドメイン間通信では、APIはサービスコンシューマのために最適化されなければなりません。このため、読み出し最適化設計の原則に従わなければなりません。

このようなドメイン境界を設定する際、データプロダクト管理や、異なるドメイン間でのデータの共有方法と重なる部分が多いことに気づくかもしれません。内部的なビジネスロジックとデータは内部的なものであり、ほかのドメインに公開すべきではありません。ドメイン境界を越える場合は、APIの設計や登録方法について、さまざまな原則が存在するので適用しなければなりません。

5.5　エコシステム通信

1章では、多くの組織がAPIベースやWebベースの新しいビジネスモデルを模索していることを紹介しました。あらゆる業界の企業が、ほかのデジタル企業と密接に連携しなければ、事業の拡大や、イノベーション、破壊的革新、単純な

競争力強化すらできない、という結論に達しました。このトレンドは今後も続くと予想され、アーキテクチャは、サービスコンシューマとの**内部**（サービス）通信と**外部**（サービス）通信を明確に分けなければならなくなるでしょう[†16]。これには、以下のような3つの重要な理由があります。

[†16] ここでいう内部とは、内部アプリケーションのAPIを指すのではありません。内部とは組織や企業の境界内を意味します。

- 組織が内部的に使う言語は、社内では会社のコントロール下にありますが、社外ではそうでないことが多いのです。API設計の形式や構造は、ほかのパーティや法規制によって決められることがよくあります。改訂された決済サービス指令（PSD2）は、欧州の銀行セクターに対して、口座、取引先、取引内容などによって、APIを厳密に分類するよう義務付けています。このような理由により、既存の内部サービスを再利用する際に、翻訳処理を追加しなければならなくなることがよくあります。
- パブリックWebに公開されるAPIは、外部パーティが必ずしも信頼できるとは限らず、相手が誰かもわからないので、さらなる注意が必要です。APIを安全に保護するためには、APIの監視やスロットリングを行ったり、安全なアクセスのために、IDサービスプロバイダーを用意したりする必要があります。
- ビジネス上の合意書に基づいてスロットリング（呼び出しの量を制限すること）するのが良いでしょう。商用で提供されている外部APIには消費プランがあり、多くの場合、API呼び出し数に応じて課金されます。さまざまなAPIユーザーを区別する必要があるため、API監視は、これまでのものとは異なる場合もあります。

外部パブリックWebサービスとの通信では、アーキテクチャの内部サービスと外部コンシューマの間に、APIレイヤーを追加しなければなりません。これは2種類の調停のために必要です。外部サービスプロバイダーと企業の内部環境間の調停と、企業環境自体の内部での調停です。

内部と外部の抽象化は、同じAPIレイヤーで実装できるのでしょうか。技術的には、内部と外部の両方のトラフィックをルーティングするために、同じAPIを使うことができます。しかし、ここで、APIビジネス戦略を考慮する必要があります。オープンAPIが成功するかどうかは、外部の開発者を惹きつけるこ

とができるかどうかにかかっています。オープンAPIの目的は、開発者やユーザーが実際に使いたいと思うようなアプリケーションを開発できるようにすることです。抽象化のレイヤーを追加すれば、内部環境と疎結合になるため、より迅速にイノベーションを実現できます。

企業内部の通信と外部のトラフィックを分離する理由としては、外部環境に対しては、セキュリティ管理が異なるということも挙げられます。外部トラフィックには、ログ記録や、DDoS防御（https://oreil.ly/OjLEU）、異なるIDプロバイダーが必要になります。

外部APIやエコシステム通信は、内部と外部のサービス通信を区別する必要があるため、API戦略に大きな影響を与えるでしょう。このほかに、アーキテクチャに影響を与えるものとしては、リッチな顧客体験を提供するためのチャネルも挙げられます。次はそれについて見ていきましょう。

5.6　エクスペリエンスAPI

APIは、Webベースやオムニチャネル（https://oreil.ly/5vu6g）を介してリッチな顧客体験を提供するためにも使用されます。これらのチャネルでは顧客とのコミュニケーションが行われ、コンテンツがほかのビジネスパーティに提供されます。このような場面でのAPIは、エンドユーザーとの直接的なコミュニケーションに使用されるため、ほかのAPIと区別します。したがって扱いが異なり、システム間のやり取りよりも、人間とのやり取りが重要になります。 このやり取りは、Webサイトや、モバイルデバイス、チャットボット、ゲーム、ビデオ、音声認識システムなどを介して行われます。オンラインチャネルは企業の顔となることが多いため、すばらしいユーザーインタフェースとエクスペリエンスを提供することが重要となり、そのためにはエンタイトルメントチェック、リクエスト処理と検証、状態管理、キャッシュ、Webセキュリティなどのコンポーネントをアーキテクチャに追加する必要があります。これらのコンポーネントは一般的に「デジタルエクスペリエンス」ドメインの一部となります。

これらのコンポーネントをエンドユーザーに近いドメインで管理する理由は、ユーザーエクスペリエンスを向上すること自体が重要な仕事だからです。フォーム検証などのWebアプリケーションのセキュリティは、APIのセキュリティとは異なります。また、ユーザーエクスペリエンスは高速である必要があるので、APIの対話処理やキャッシュの方法は大きく変更されることになります。

サーバー側のリソースに対して、たくさんのAPI呼び出しを行えば、ユーザーエクスペリエンスが大幅に低下します。例えば、国を選択するためのリストは、毎回APIゲートウェイを介してネットワーク経由で取得するよりも、エンドユーザーの近くにある専用の（キャッシュ）コンポーネントに保存しておく方が良いでしょう。呼び出し回数をより少なくして、より効率的に情報を取得できるようにします。このため、アーキテクチャに一連の機能を追加する必要があるのです。

5.6.1　GraphQL

一貫したAPIエクスペリエンスを提供するための代表的なコンポーネントが、GraphQLサービス（https://oreil.ly/JB4OA）です。GraphQLは既存APIの付加レイヤーとして機能します。GraphQLは、複数のAPIから必要なフィールドを選択するためのクエリ言語です。GraphQLは、さまざまなシステムから膨大な量のデータをリアルタイムで抽出するのに適したソリューションです。

GraphQLでは、**スキーマフェデレーション**（https://oreil.ly/zeyJy）が利用できます。つまり、スキーマを単一のデータグラフとして組み立てた上で、すべてのデータソースに対してクエリを実行するような統一インタフェースが使えます。繰り返しになりますが、エンタープライズデータモデルや中央ゲートウェイモデルを作成する、というような失敗をしないようにしてください。もっと詳しく知りたい方は、Thomas Frisendal氏の“Visual Design of GraphQL Data: A Practical Introduction with Legacy Data and Neo4j”（Apress）を是非読んでみてください。

APIアーキテクチャにおけるGraphQLサービスは、ドメイン内のみに配備することも、ドメイン間通信のための機能として配備することもできます。後者の場合は、2章で説明した原則すべてに従う必要があります。すべてのGraphQL

エンドポイントは、設計やガバナンスについて同じ原則に従わなければなりません。

5.6.2　フロントエンドのためのバックエンド

ユーザーエクスペリエンスを向上させ、フロントエンドクライアントのニーズに応えるには、レイヤーやコンポーネントを追加して、ユーザーインタフェースやエクスペリエンス関連のロジック処理を行うという方法もあります[†17]。このレイヤーは、ある特定のフロントエンド向けに最適化できます。例えば、モバイルアプリケーションやシングルページアプリケーションでは、デスクトップベースのWebトラフィックとは異なる最適化が行われ、異なるコンポーネントが必要になる場合があるでしょう。このレイヤーでは、GraphQLを利用したくなるかもしれません。

[†17] Sam Newman氏は、共通の表現を作ったり密結合にしたりすることなく、サービスを活用して、最適化と抽象化を適用する設計思想のことを、「フロントエンドのためのバックエンド」（https://oreil.ly/cxfW6）と表現しています。

これまで見てきたように、APIを介してドメインを相互作用させるには、さまざまなパターンと考慮すべき点があります。どのように組み合わされるかを示すために、次の節では高レベルのAPIアーキテクチャを紹介し、さまざまなドメインがどのように相互作用するかを説明します。この例を一通り見た後、API管理におけるメタデータの役割を簡単に紹介します。

5.7　実例

今後数年間で、企業は無数の新しいアプリケーションを提供することになるでしょう。その中心となるのがAPIです。APIは最新型の「正面玄関」を実現し、アプリケーション間の高度な統合を可能にします。スケーラビリティを確保するには、ドメイン指向設計をアーキテクチャに適用する必要があります。理想的には、それぞれのチームがアプリケーションオーナーシップを持ち、ほかのドメインからスムーズな消費を可能とするAPIセットを提供するようにします。

図5-6に示すアーキテクチャでは、この章を通して説明したいくつかのパターンを使用しています。ドメインA、B、Cでは、原則は同じです。それぞれのドメインはAPIを公開するためにAPIゲートウェイを使用します。これらのAPIゲートウェイはAPIガバナンスの管理下にあり、すべてのセキュリティ標準とそのほかの原則を遵守するようにします。

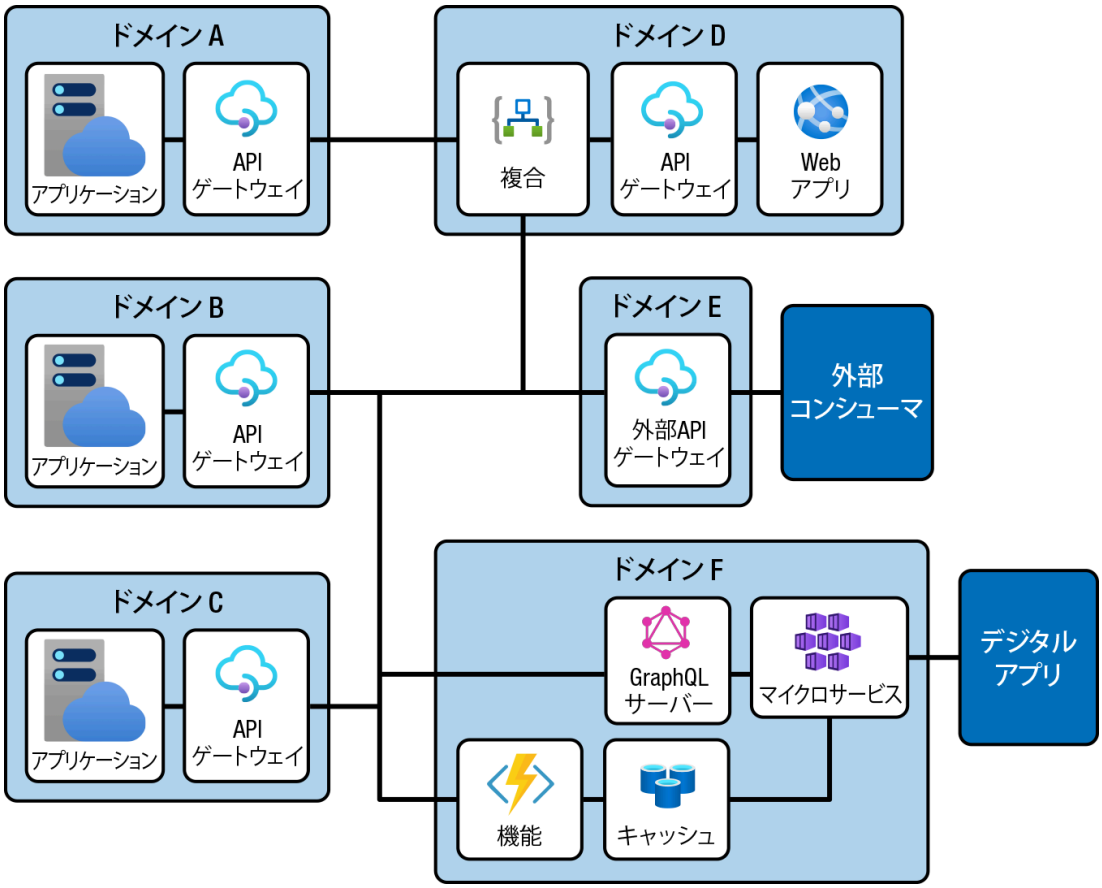

図5-6　さまざまなドメインがAPIをどのように提供し、ほかのドメインからどのようにAPIを利用しているかを示すAPIアーキテクチャの例

ここで説明するアーキテクチャは、説明のために単純化されています。実際には、ファイアウォール、ロードバランサー、アプリケーションゲートウェイ、仮想ネットワークなどが存在します。

ドメインD、E、Fは消費ドメインです。これらはAPIを利用して価値を提供したり、ほかのパーティと協力してイノベーションを実現したりします。図5-6

でわかるように、それぞれのドメインはさまざまな方法でAPIを利用しています。例えば、ドメインDは一連のリクエストを単一のリクエストとして実行する複合APIを開発しています。新たに作成されたエンドポイントは、ドメイン自身によってWebベースのアプリケーションの中で使用されます。ドメインEは外部APIゲートウェイパターンを使っています。このレイヤーは、外部コンシューマ向けにAPIを翻訳し、セキュアにするために必要です。ドメインFは、オンライン上の顧客にリッチなエクスペリエンスを提供する大規模なドメインです。バーチャルエージェント、チャットボット、分析サービスなどの機能を提供します。ドメインFのアーキテクチャでは、マイクロサービス設計が適用され、セッション状態情報を保存するためのキャッシュが使用されます。ほかのドメインからの情報でデータをリッチ化するために、同じ保存場所が使われます。ほかのAPIを呼び出し、その結果をキャッシュストアに格納する機能を持ちます。このアプローチによって、さらにリッチな顧客体験が実現できます。最後に、ほかのドメインへのリクエストを絞り込むためにGraphQLサービスが使用されます。

このアーキテクチャはシンプルに見えますが、これまでに説明したアーキテクチャの原則とガイドラインをすべて守ることが極めて重要です。例えば、すべてのドメイン境界は明確に設定され、複雑なロジックはAPIゲートウェイの後ろに隠蔽されなければなりません。ロジックの管理も同様です。複合APIの場合、一連のAPI呼び出しを構築するためにロジックが必要です。これには固有のビジネス知識が含まれるため、この知識を所有するドメイン境界内で、この複合コンポーネントを管理するのが最適です。APIの公開と登録にも同じ原則を適用する必要があります。これにより、導入を早め、習得を容易にします。

5.8　メタデータ管理

メタデータ管理はAPI管理にとって重要な側面です。APIの数が増えるにつれて、ドメインをサポートして、サービスを検索可能にし、再利用性を高めることが不可欠になります。ベストプラクティスは、属性や説明を含むすべてのAPIを中央リポジトリまたは開発ポータルで公開することです。一般的に、商用のAPI管理プラットフォームにはそのようなリポジトリがあります。もしも、商用のベンダーを使いたくない場合は、オープンソースのAPI管理ソリューションも使えます。OpenAPI.Tools（https://openapi.tools）では、高品質で最新のオープンソースツールの一覧を管理しています。APIを記述するためにOpenAPI Initiative（https://www.openapis.org）のベンダー中立フォーマットを使うのも良い考えです。

マイクロサービスとメタデータ

マイクロサービスに関するインサイトを提供するためにメタデータは必要でしょうか。もちろんです。何千ものマイクロサービスが大規模なプラットフォーム上で実行されることがあります。誰がどのサービスを所有し、何を行い、どのようなデータを生成しているかを知ることは重要です。マイクロサービスが自分自身を発行できるようにするには、ランタイムにメタデータラベルを追加するか、マイクロサービスが配備された後にAPIを呼び出すようにしてください。チームにとって、より簡単にするには、API呼び出しをコンテナイメージの一部にしても良いでしょう。このほか、チームを支援するために、コードのスニペットやスケルトンテンプレートを提供する方法もあります。

複数のAPIゲートウェイ、ESB、サービスメッシュが導入されている分散環境では、すべてのAPIを把握しておくことが特に重要です。それぞれの統合機能が中央リポジトリに接続されている場合にのみうまくいきます。どのサービスがどのドメインに属し、どのドメインが相互作用しているかを知ることができるからです。この情報は、ランドスケープ全体を管理する上で大きなメリットとなります。

5.9　データプロダクトを提供する読み出し指向API

APIは単独で存在するわけではありません。APIはデータプロダクトアーキテクチャの中でも使うことができます。例えば、CQRS（「4.2.1　CQRSとは何か」参照）パターンを実装して、読み出し専用データへのリアルタイムアクセスを提供することができます。同じ境界内で、強一貫性のあるデータ読み出しと、 結果一貫性のあるデータ読み出しを分けることもできます。この仕組みを説明しましょう。この設計におけるAPIゲートウェイは、API呼び出しを運用システムやデータプロダクトアーキテクチャに転送することで、ルーターの役割を果たします。具体的に説明すると、このような設計では、「GET rest/v1/resources/1」のような結果一貫性のある読み出しリクエストはすべて読み出しストアにルーティングされます。「POST rest/v1/resources」のようなコマンドは、すべてコマンドストアにルーティングされます。このような設計であれば、コンシューマはその違いに気づかないでしょう。コンシューマが知らなければならないのは、通信のエンドポイントだけです。インテリジェントなルーティングは提供ドメイン内部で行われます。

また、データプロダクトを設計する方法としては、データが直接消費されるキャッシュとして機能する、読み出し指向のマイクロサービスを配備する方法があります。このような設計では、それぞれのマイクロサービスが自身のデータストアに、ある特定のデータプロダクトを保持し、APIエンドポイントでこのデータプロダクトを提供します。それぞれのマイクロサービスがある特定のリソースのデータをキャッシュする、リソース指向アーキテクチャと同様です。この場合も、APIゲートウェイはすべてのコマンドとクエリの分離を行います。このような読み出し指向のマイクロサービスを使用する設計は、データメッシュアーキテクチャにおけるデータプロダクトの概念に近いものです。

3章で学んだように、CQRSを使えば、アーキテクチャのスケーラビリティと性能を大幅に向上させることができます。クエリを、より効率的に処理するように最適化できます。これは、ネットワークの片側に運用システムがあり、反対側にクエリするシステムがあるような分散環境でも有効です。クエリの負荷が非常に高い場合、読み出し集中型のデータ課題に対する唯一の解決策が、CQRSです。

5.10　まとめ

急速に変化する企業環境において、本章で学んだAPI管理とベストプラクティスは、API戦略を刷新することでしょう。サービス指向アーキテクチャの基本となる部分は変更せずに、より責任が明確になり、最新のパターンと原則が使われ、データ管理とより強く連携し、そしてより柔軟なアーキテクチャになります。APIファースト戦略への転換を計画している場合、以下に挙げるようなベストプラクティスの採用を検討してください。

- API戦略を構築し、ビジネスアーキテクチャと整合させます。それぞれのビジネスケイパビリティ内での使用状況を確認し、どのビジネスケイパビリティがほかのどのビジネスケイパビリティと統合されているかを明らかにします。
- APIの設計と構造を開発することは、アーキテクチャに関する重要な作業です。ほかのドメインと連携するAPIを定義する際には、APIの設計をデータプロダクトの設計と整合させましょう。APIとデータプロダクトのベースとなるアプリケーションデータベースは常に同じであることを忘れないでください
- 内部アプリケーション開発のためのAPIと、ほかのドメインとの連携のためのAPIのガイダンスを分けてください。例えば、内部APIは技術的であって

もかまいませんが、ほかのドメインに提供されるAPIは安定していて、説明がわかりやすく、利用しやすいものでなければなりません。

- プラットフォームとサービスの標準を設定し、APIが果たす範囲と役割に整合させます。例えば、外部のサービスコンシューマに対するAPIは、高まるセキュリティ要件を満たすために、別のAPIゲートウェイで公開する必要があります。
- API設計のガイドラインとベストプラクティスを公開しましょう。このコンテンツはAPIコミュニティが所有するようにしましょう。
- マイクロサービスアーキテクチャに留意しましょう。何百ものサービスの複雑さに直面するチームをそのまま放置するのではなく、境界を追加することを検討してください。

APIはRESTデザインであるため、APIを使用することで、すべてのデータを分散管理することができます。複数のAPIゲートウェイとサービスメッシュを同時に配備し、チームやドメインごとに独自のAPIゲートウェイを使用してサービスを提供することも可能です。メタデータリポジトリを使用することで、どのサービスが利用可能で、どのように消費されるかについての情報を保持し、コントロールしつづけることができます。

APIアーキテクチャのモジュール性とリクエスト/レスポンスモデルにより、ある特定のユースケースに対して、高度にモジュール化されたマイクロソリューションも実現できます。これはアーキテクチャを非常にスケーラブルにします。ここで注意すべき点は、ドメインアーキテクチャと同じように、2章で説明した設計原則に従わなければならない、ということです。APIは、リアルタイムでデータを生成し、送受信するために使われるプロダクトと見なすべきです。それらは、読み出しやすく簡単に消費できるように責任を持って管理され、最適化されなければなりません。APIは、そのドメインのユビキタス言語を使用すべきであり、ほかのドメインのビジネスロジックを取り込むべきではありません。

次章では、非同期メッセージングとイベント駆動型通信に焦点を当て、そこからどのような知見が得られるかを見ていきます。